
Lab Report - Exercise 3

Amir Khosravinejad
(a.khosravinejad@edu.rptu.de)

Patrick Pfau
(ppfau@rptu.de)

Group 5

June 11, 2026

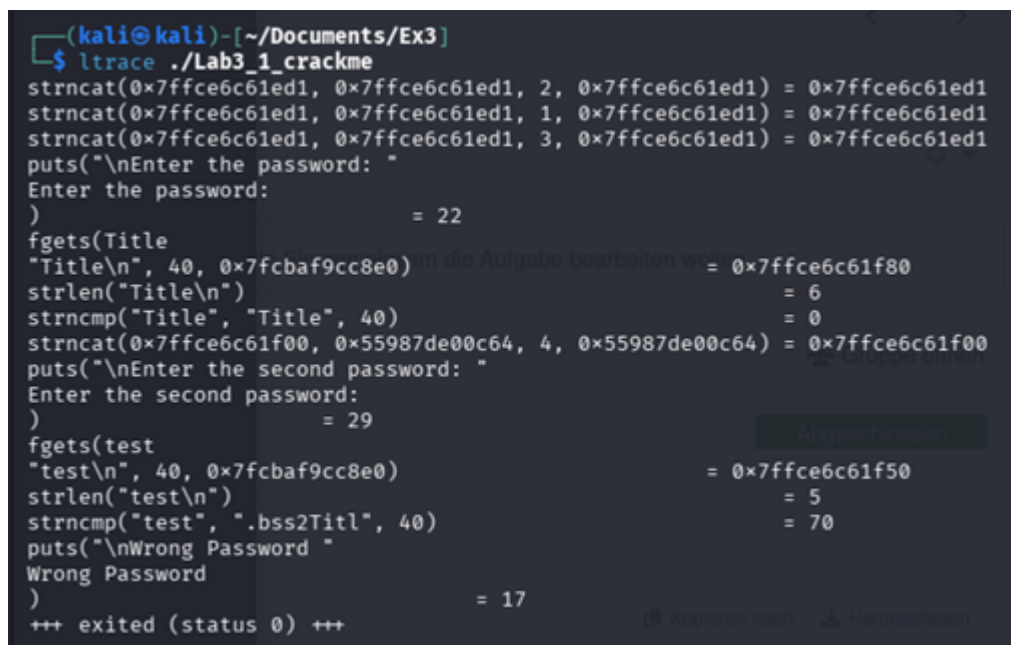
1 Exercise 1

First Password

We printed out strings of the vulnerable file with **strings Lab3_1_crackme** and just tried out "Title" for the first password.

Second Password

The second password ".bss2Titl" can be found out by typing in **ltrace ./Lab3_1_crackme** using the tool **ltrace**.



```
(kali㉿kali)-[~/Documents/Ex3]
$ ltrace ./Lab3_1_crackme
strncat(0x7ffce6c61ed1, 0x7ffce6c61ed1, 2, 0x7ffce6c61ed1) = 0x7ffce6c61ed1
strncat(0x7ffce6c61ed1, 0x7ffce6c61ed1, 1, 0x7ffce6c61ed1) = 0x7ffce6c61ed1
strncat(0x7ffce6c61ed1, 0x7ffce6c61ed1, 3, 0x7ffce6c61ed1) = 0x7ffce6c61ed1
puts("\nEnter the password: ")
Enter the password:
) = 22
fgets(Title
"Title\n", 40, 0x7fcbaf9cc8e0) = 0x7ffce6c61f80
strlen("Title\n") = 6
strncmp("Title", "Title", 40) = 0
strncat(0x7ffce6c61f00, 0x55987de00c64, 4, 0x55987de00c64) = 0x7ffce6c61f00
puts("\nEnter the second password: ")
Enter the second password:
) = 29
fgets(test
"test\n", 40, 0x7fcbaf9cc8e0) = 0x7ffce6c61f50
strlen("test\n") = 5
strncmp("test", ".bss2Titl", 40) = 70
puts("\nWrong Password ")
Wrong Password
) = 17
+++ exited (status 0) +++
```

Third Password

The third password is "test.bssteT". With **gdb ./Lab3_1_crackme** we open the debugger and can print the whole file with **layout asm** and see the important function "checker". Then, we set a breakpoint on the "checker" function (**b checker**) and run the code with **r**.

```

(kali㉿kali)-[~/Documents/Ex3]
$ gdb ./Lab3_1_crackme
GNU gdb (Debian 17.1-4) 17.1
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word" ...
Reading symbols from ./Lab3_1_crackme ...
(No debugging symbols found in ./Lab3_1_crackme)
(gdb) b checker
Breakpoint 1 at 0xb66
(gdb) r
Starting program: /home/kali/Documents/Ex3/Lab3_1_crackme
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Enter the password:
Title

Enter the second password:
.bss2Titl

Enter the third password:
test

```

When we reach the breakpoint we check out the registers "rdi" and "rsi" with **info registers rdi rsi**. Finally, we read out the register: **x/s \$rsi** and receive the last password.

```

Breakpoint 1, 0x000055555400b66 in checker ()
(gdb) info registers
rax          0x0          0
rbx          0x0          0
rcx          0x7fffffffdc10 140737488346128
rdx          0x28         40
rsi          0x7fffffffdc10 140737488346128
rdi          0x7fffffffdc50 140737488346192
rbp          0x7fffffffdbf0 0x7fffffffdbf0
rsp          0x7fffffffdbf0 0x7fffffffdbf0
r8           0x555555603725 93824992950053
r9           0x0          0
r10          0x0          0
r11          0x202        514
r12          0x1          1
r13          0x7ffff7ffd000 140737354125312
r14          0x7ffff7ffde18 140737488346648
r15          0x0          0
rip          0x555555400b66 0x555555400b66 <checker+4>
eflags      0x202        [ IF ]
cs           0x33         51
ss           0x2b         43
ds           0x0          0
es           0x0          0
fs           0x0          0
gs           0x0          0
fs_base     0x7ffff7daa740 140737351690048
gs_base     0x0          0
(gdb) info registers rsi rdi
rsi          0x7fffffffdc10 140737488346128
rdi          0x7fffffffdc50 140737488346192
(gdb) x/s $rdi
0x7fffffffdc50: "test"
(gdb) x/s $rsi
0x7fffffffdc10: "test.bssteT"

```

To gain the flag we run the program and receive it: Flag: 5820585582.

Exercise 2

Decoding secret1.txt file:

Using some decoders online, we found out this text file(secret1.txt) is encoded via base64 encoding. So the decoded text is: Encrypting_Your_Data_Safely

Decoding secret2.txt file:

Using [hashes.com](https://www.hashes.com/), this file is encoded hex. So using a hex decoder, the result is: `changecalculate_5To:Hex(Res+=0xba5eba11)`. So it says that we should change the binary of fifth subcalculation and add hexadecimal encoded of "Res+=0xba5eba11" which is: 5265732B3D30786261356562613131

Meaning of calculation functions

We renamed all variable names in Ghidra to make the code easier to read. To understand the meaning of each calculation function we tried to read the code decompiled in Ghidra (AI helped us).

calculate_1

It lowercases every character. It can be seen in Ghidra where it loops over the word and lowercases each character one by one.

calculate_2

The function iterates through the whole string and adds the ASCII code for each character and outputs that sum.

calculate_3

The function takes 2 parameters. The first is added to the length of the second.

calculate_4

This function gets 2 parameters and checks if the second parameter is a palindrome. If yes, 100 will be multiplied to the parameter 1, otherwise it'll be multiplied by 50.

calculate_5

The unmodified version: Function 5 returns the input variable with the actual timestamp (in seconds).

Modified version: As in the secret2.txt file implies, we should change it to not add current time stmap but add it by 0xba5eba11.

calculate_6

The last function checks whether the characters of the string are sorted in the right order. If so, the result is input parameter XOR 0xdeadbeef, otherwise the in put parameter is returned unchanged.

calculate_last

This function checks if the string has at least one vowel.

Finding the flag

First, we tried to edit the line

```
001014ef 01 45 ec      ADD     dword ptr [RBP + local_1c],EAX
```

of the program file communicate_secret in Ghidra, as said in the decrypted message of secret2. With patch instruction we could resolve the problem. Next, we coded the calculation functions in python. Using the decoded string "Encrypting Your Data Safely" of secret1 we receive the flag 0xba60f462

```
1 import time
2 import string
3
4 def calculate_1(input_string):
5     result = input_string.lower()
6     print(f"[*] The result of the first computation is: {result}")
7     return result
8
9 def calculate_2(input_string):
10    checksum = sum(ord(c) for c in input_string)
11    print(f"[*] The result of the second computation is: {checksum}")
12    return checksum
13
14 def calculate_3(input_value, input_string):
15    result = input_value + len(input_string)
16    print(f"[*] The result of the third computation is: {result}")
17    return result
18
19 def calculate_4(input_value, input_string):
20    if len(input_string) == 1:
21        result = input_value * 50
22        print(f"[*] The result of the fourth computation is: false, {result}")
```

```

23     else:
24         reversed_string = input_string[::-1]
25         if input_string == reversed_string:
26             result = input_value * 100
27             label = "true"
28         else:
29             result = input_value * 50
30             label = "false"
31         print(f"[*] The result of the fourth computation is: {label}, {result}")
32     return result
33
34 def calculate_5(input_value):
35     result = input_value + int(0xba5eba11)
36     print(f"[*] The result of the fifth computation is: {result}")
37     return result
38
39 def calculate_6(input_value, input_string):
40     is_sorted = (''.join(sorted(input_string)) == input_string)
41     if is_sorted:
42         result = input_value ** 0xdeadbeef # Potenz!
43     else:
44         result = input_value
45     print(f"[*] The result of the sixth computation is: {result}")
46     return result
47
48 def calculate_last(input_value, input_string):
49     contains_vowel = int(any(c in 'aeiou' for c in input_string))
50     result = input_value + contains_vowel
51     print(f"[*] The result of the last computation is: " + hex(result))
52     return result
53
54 def main(input_string):
55     if len(input_string) >= 0x29:
56         print("\n[+] Make sure the string is 40 characters long or less!\n")
57         return 1
58
59     input_string = calculate_1(input_string)
60     result = calculate_2(input_string)
61     result = calculate_3(result, input_string)
62     result = calculate_4(result, input_string)
63     result = calculate_5(result)
64     result = calculate_6(result, input_string)
65     calculate_last(result, input_string)
66     return 0
67
68 if __name__ == "__main__":
69     import sys
70     if len(sys.argv) != 2:
71         print("\n[+] Please give a string as an input! (not more than 1 argument)\n")
72         sys.exit(1)
73     sys.exit(main(sys.argv[1]))

```